**Primer módulo Python que consume un servicio externo (Bloque 5) / Un error de red a propósito, para aprender a manejarlo**

✓ **Hecho:** Ver

📘 **UNIDAD U1.3** · Python aplicado para integración con IA

# Un error de red a propósito, para aprender a manejarlo

Ya viste los conceptos por separado: qué es un módulo, por qué las funciones importan, y cómo se estructura una respuesta JSON.

En esta lección, se muestra funcionando juntos, construyendo un módulo real de consumo de servicios, y hace algo más: provoca un error a propósito, para que veas con tus propios ojos qué pasa cuando algo falla y no está previsto.

Esta lección describe ese recorrido, como referencia para cuando lo veas en vivo y para cuando construyas tu propia primera versión más adelante en este mismo bloque.

> El ejemplo de esta lección usa la misma librería **"requests"** que ya instalaste en la **Unidad 1.2** (con tu entorno virtual activo). Si sigues este recorrido en tu propio computador, no necesitas instalar nada nuevo para reproducirlo.

---

## 🛠️ Qué se construye en esta demostración

🟢 **El punto de partida es simple**: una función que recibe los datos necesarios para consumir un servicio externo (la URL y lo que haga falta enviar), envía la petición, y devuelve el resultado ya procesado, no la

externo (la URL y lo que haga falta enviar), envía la petición, y devuelve el resultado ya procesado, no la respuesta cruda.

**Esa última parte importa**: quien use esta función después no debería tener que saber nada sobre cómo se hizo la petición ni cómo viene estructurada la respuesta original, solo debería recibir el dato que le interesa, ya extraído y listo para usar. Esa es, en esencia, la responsabilidad de un módulo bien diseñado: esconder la complejidad de cómo se hace algo, detrás de una función simple de usar.

> Este **patrón** (función que recibe lo necesario, entrega el resultado ya extraído, y esconde todo lo demás) no es un detalle exclusivo de la demostración. Es el mismo criterio que vas a aplicar tú, línea por línea, cuando construyas tu propia primera versión de este módulo más adelante en este bloque, y es también el mismo criterio que se revisa en la rúbrica de calidad de código al final de la unidad.

---

## ⚠️ Por qué un error de red, específicamente

**Se elige provocar un error de red**, específicamente, porque es, de lejos, el **más fácil de producir a propósito frente a un grupo**: basta con apuntar la petición a una dirección que no existe o simular que la conexión no está disponible, sin necesitar que ningún servicio externo coopere con la demostración.

También es el más común en la práctica. De los tres tipos de error que vas a aprender a manejar formalmente en el Bloque 7 (uno de conexión, otro de formato de la respuesta, y otro de un dato faltante dentro de esa respuesta), el de conexión es el que menos depende de circunstancias externas para poder mostrarlo. Los otros dos son igual de reales, pero más difíciles de provocar de forma controlada, frente a un grupo, sin margen para que algo salga distinto a lo planeado. Vas a verlos con calma, uno por uno, cuando construyas el manejo completo de tu propio módulo.

---

## 🚨 El error provocado a propósito

Con esa primera versión funcionando, el instructor rompe algo deliberadamente: cambia la dirección del servicio a una que no existe, o simula que la conexión no está disponible.

🔴 El **resultado es un error de red que el programa no sabe manejar**, y el efecto es contundente: el programa se detiene por completo, en el punto exacto donde ocurrió el problema, y en pantalla aparece un mensaje técnico (un rastro de en qué línea y por qué falló) que no significa nada para alguien que esté usando la aplicación, solo para quien esté mirando el código en ese momento.

> **Esto no es un accidente de la demostración, es el punto central de ella**. Un programa que se detiene por completo apenas algo sale mal es frágil: útil solo mientras todo funcione perfectamente, y sin ningún valor cuando el problema es exactamente el que más importa anticipar, como una conexión que falla en medio de una clase, una **API** que está temporalmente caída, o una respuesta

conexión que falla en medio de una demo, una **API** que está temporalmente caída, o una respuesta que llega en un formato distinto al esperado.

> ℹ️ **Una aclaración rápida**: un error de sintaxis (por ejemplo, olvidar un paréntesis) impide que el programa arranque siquiera, y aparece antes de que se ejecute cualquier línea. Un error de conexión, como el de este ejemplo, es distinto: el programa sí arranca y corre con normalidad hasta el momento exacto en que intenta comunicarse con algo externo, y recién ahí falla.

En términos de código, la llamada que falla se ve tan simple como esta, sin nada que anticipe el problema:

```
resultado = obtener_datos(url_rota)
print(resultado)
```

Si quieres seguir la demostración en tu propia terminal (no es obligatorio: tu propia primera versión la construyes recién en la siguiente lección), puedes guardar esto en un archivo, por ejemplo "**test_conexion.py**":

```
import requests

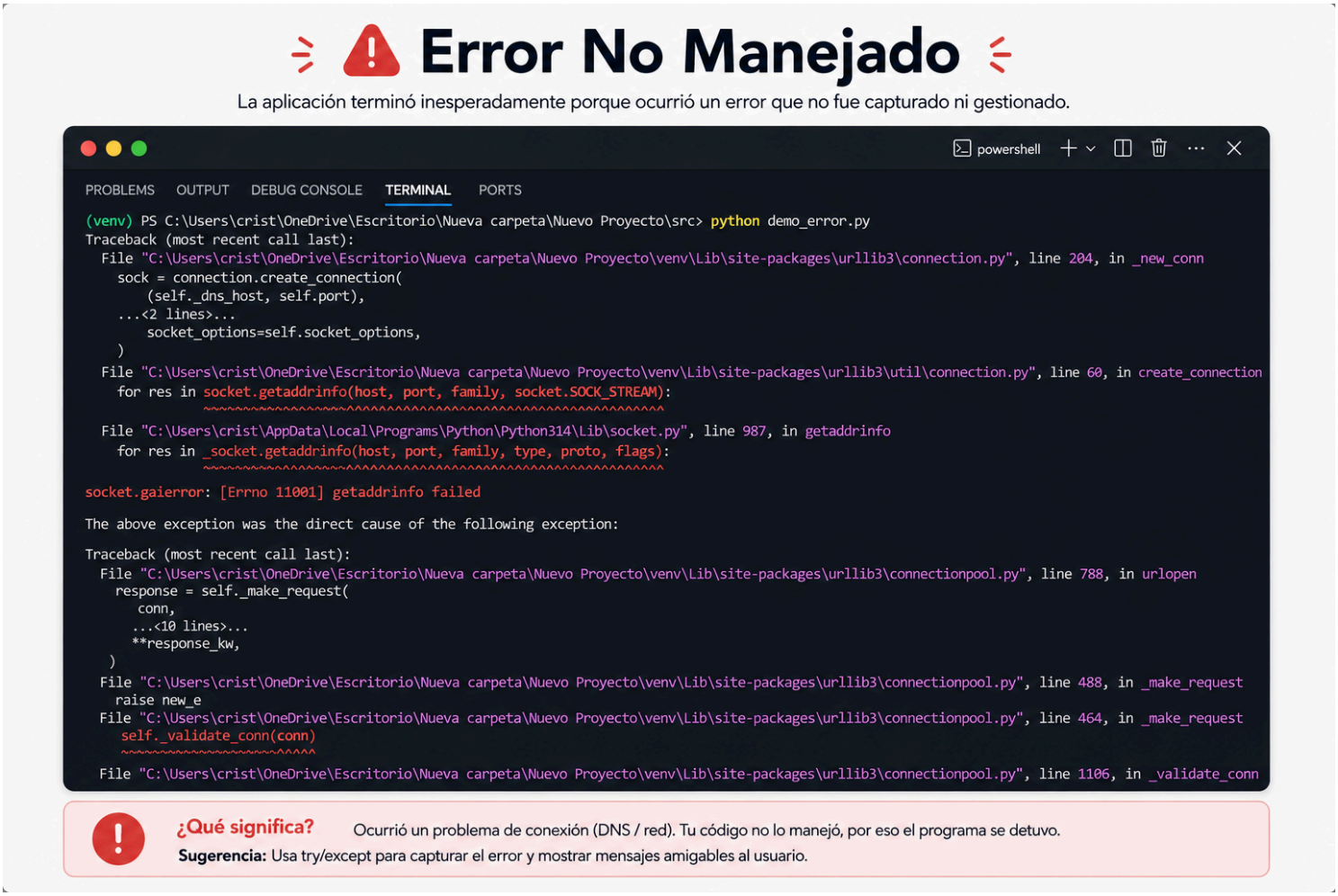
def obtener_datos(url):
    respuesta = requests.get(url)
    return respuesta.json()

url_rota = "https://esta-direccion-no-existe.ejemplo/datos"
```

*Esa segunda línea nunca llega a ejecutarse: el programa se detiene por completo en la primera, con un rastro técnico en pantalla (un "traceback") que señala la línea exacta y el tipo de error, pero que no le sirve de nada a quien esté usando la aplicación.*

*El rastro completo (el "**traceback**") varía según tu sistema operativo y las versiones instaladas, pero siempre termina en una línea como esta, que es la que realmente hay que leer:*

```
Traceback (most recent call last):
  ...
requests.exceptions.ConnectionError: HTTPSConnectionPool(host='esta-direccion-no-existe.eje
```




---

## El manejo controlado: qué cambia

🟢 **La segunda parte de la demostración muestra la solución**: envolver la parte riesgosa del código (la que depende de algo externo, como una conexión de red) en una estructura que anticipa que algo puede salir mal, y define de antemano qué hacer si eso ocurre. En vez de dejar que el programa se detenga sin control, esa estructura intercepta el problema en el momento en que ocurre, y le da al programa la oportunidad de responder de una forma pensada: devolver un mensaje claro sobre qué falló, registrar el problema para que quede evidencia de que ocurrió, o intentarlo de nuevo, según lo que tenga sentido en ese caso.

La diferencia entre las dos versiones no está en si el error ocurre (el error de red ocurre en ambas), está en qué pasa después.

➡️ En la primera versión, el error termina el programa.

➡️ En la segunda, el error queda contenido: el programa sigue funcionando, y quien lo esté usando recibe una respuesta entendible en vez de un mensaje técnico sin sentido para él.

La misma llamada, envuelta en esa estructura que anticipa el problema, se ve así:

```
try:
     resultado = obtener_datos(url_rota)
 except Exception as error:
     resultado = None
     print("No se pudo completar la petición:", error)
```

> Este es solo un adelanto conceptual de lo que vas a construir a fondo en el Bloque 7 (con excepciones específicas, no genéricas, y una respuesta pensada para cada caso). Por ahora, lo que importa es ver que el programa ya no se detiene: el error queda contenido dentro del "except", y el resto del código sigue corriendo.

Esta vez el programa no se detiene: en tu consola aparece un mensaje que tú mismo definiste, no un traceback:

```
No se pudo completar la petición: HTTPSConnectionPool(host='esta-direccion-no-existe.ejempl
```

Vale la pena notar algo sobre ese ejemplo, para que no lo copies tal cual más adelante: usa "**except Exception**", la forma más genérica de capturar un error en **Python**, que atrapa prácticamente cualquier cosa que salga mal, no solo el error de red que se está demostrando aquí.

Es una elección deliberada para esta demostración puntual, no una recomendación para tu propio código. Capturar de forma tan amplia esconde también errores que no anticipaste, incluyendo bugs reales en tu programa, como si fueran parte del comportamiento esperado. En el **Bloque 7** vas a aprender a capturar excepciones específicas (una para el error de conexión, otra distinta para un problema de formato), precisamente para evitar esconder algo que en realidad necesitabas ver.




---

## 🧠 Por qué provocar el error a propósito, en vez de solo explicarlo

Sería más rápido simplemente describir en palabras qué es el manejo de errores, sin mostrar el antes y el después. Pero ver un programa detenerse abruptamente, con un mensaje técnico en pantalla, deja una impresión distinta a la de solo escuchar que "es importante manejar errores": es la diferencia entre saber algo en abstracto y haberlo visto fallar de verdad.

Esa impresión es intencional, y es la misma razón por la que, más adelante en esta unidad (en el Bloque 7), te va a tocar incorporar tú mismo este mismo tipo de manejo a tu propio módulo, no simplemente leer sobre cómo hacerlo.

> 🚀 Hay además un **beneficio práctico, más allá de lo pedagógico**: acostumbrarte a leer un mensaje de error técnico sin que te genere pánico es, en sí mismo, algo que se entrena. La primera vez que un programa se detiene con un rastro largo y en apariencia incomprensible en pantalla, es fácil sentir que algo se rompió de forma grave e irreversible.
>
> Con la práctica, **ese mismo mensaje se convierte en información útil, no en una señal de alarma**: te dice con precisión en qué línea ocurrió el problema y de qué tipo es.

Cuanto antes pierdas ese temor inicial frente a un error en pantalla, más rápido vas a poder diagnosticar problemas reales durante el resto del programa, en vez de quedarte paralizado frente al primer mensaje

técnico que no reconozcas.

## Siguiente actividad

Construcción del módulo · Actividad práctica

Con este recorrido ya visto (un módulo simple funcionando, un error provocado a propósito, y la diferencia que hace manejarlo), la siguiente parte de esta clase es donde **construyes tú mismo tu primera versión de este módulo**, consumiendo un servicio de prueba.

Última modificación: jueves, 24 de septiembre de 2026, 12:39